

```
1 function [spherePos90fm,TS_theor_binned,calibration] =✓
calibrateSig100(Config,Data,sphereProp,WaterProp,TS_theor_params)
2 % Calibrate Nortek Signature100 echosounder
3 % G. Cutter & S. Nylund, 2019
4
5
6 % % %% Load data for debugging in stand-alone mode
7 % % clear; close all; clc;
8 % % load('calibrateSig100wkspc.mat')
9
10
11 %% OPTIONS
12 DEBUGGING      = true;
13 doSHOWFIGS     = true;
14 doPC           = true;
15
16 DEBUGEXPERIMENTAL = false;
17 doRAWSAMPLE90ECHOGRAM = true;
18
19 % OFFSETS
20 TARGSAMPOFFSET  = 43;
21 doTARGOFFSET    = true;
22 % OFFSETDIST120 = -2.4; % (m) offset for sphere distance in 120
23 OFFSETDIST120   = 0; % no longer needed, it seems
24
25 %% GLOBALS & INITs
26 % water properties
27 if( isempty(WaterProp) )
28     WaterProp.soundVel      = 1500.0; % (m/s)
29     WaterProp.temp          = 14.0;   % (C)
30     WaterProp.sal           = 34.0;   % (psu)
31     WaterProp.density       = 1033.0; % kg/m^3
32 end
33
34 % sphere properties
35 if( isempty(sphereProp) )
36     sphereProp.sphereDiameter = 25.4; % mm
37     sphereProp.sphereType     = 'WC'; % 'WC' or 'CU' currently✓
supported
38     sphereProp.approxDist     = 4.6;  % m
39 end
```

```
40
41
42 if( isempty(TS_theor_params))
43     fprintf('TS_theor_params is empty. Returning.\n');
44     return;
45 end
46
47 doCALIBRATE90FM = true;
48 doCALIBRATE70   = true;
49 doCALIBRATE120  = true;
50 doRANGEBINNING  = true;
51
52 %
53 if(isfield( TS_theor_params, 'TS_theor_paramsFM'))
54     TS_theor_paramsFM = TS_theor_params.TS_theor_paramsFM;
55 else
56     doCALIBRATE90FM = false;
57 end
58 if(isfield(TS_theor_params, 'TS_theor_params70'))
59     TS_theor_params70 = TS_theor_params.TS_theor_params70;
60 else
61     doCALIBRATE70 = false;
62 end
63 if(isfield(TS_theor_params, 'TS_theor_params120'))
64     TS_theor_params120 = TS_theor_params.TS_theor_params120;
65 else
66     doCALIBRATE120 = false;
67 end
68
69 if( isempty(TS_theor_paramsFM) )
70     TS_theor_paramsFM.Ftminspec      = 68.4;    % kHz minimum✓
frequency to model
71     TS_theor_paramsFM.Ftmaxspec      = 113.4;    % kHz maximum✓
frequency to model
72     TS_theor_paramsFM.nF             = 500;      % number of✓
frequencies to model
73     TS_theor_paramsFM.sphereDiameter = sphereProp.sphereDiameter; %✓
25.4; % sphere diameter (mm)
74     TS_theor_paramsFM.sphereType     = sphereProp.sphereType; %✓
sphere type, '%WC' or 'CU'
75     TS_theor_paramsFM.HZPASS         = -0.1;     % avoids TS samples✓
```

affected by filter edges

```
76 TS_theor_paramsFM.soundVel = WaterProp.soundVel; %1500; %✓  
water sound speed
```

```
77 TS_theor_paramsFM.rhow = WaterProp.density; % water✓  
density 1033.0
```

```
78 TS_theor_paramsFM.doPLOTTSBINNED = false;
```

```
79 end
```

```
80 if( isempty(TS_theor_params70))
```

```
81 %% Define: Theoretical TS model parameters
```

```
82 TS_theor_params70.Ftminspec = 69.5; % kHz minimum✓  
frequency to model
```

```
83 TS_theor_params70.Ftmaxspec = 70.5; % kHz maximum✓  
frequency to model
```

```
84 TS_theor_params70.nF = 10; % number of✓  
frequencies to model
```

```
85 TS_theor_params70.sphereDiameter = sphereProp.sphereDiameter; %✓  
25.4; % sphere diameter (mm)
```

```
86 TS_theor_params70.sphereType = sphereProp.sphereType; %✓  
sphere type, '%WC' or 'CU'
```

```
87 TS_theor_params70.HZPASS = -0.1; % avoids TS samples✓  
affected by filter edges
```

```
88 TS_theor_params70.soundVel = WaterProp.soundVel; %1500; %✓  
water sound speed
```

```
89 TS_theor_params70.rhow = WaterProp.density; % water✓  
density 1033.0
```

```
90 TS_theor_params70.doPLOTTSBINNED = false;
```

```
91 end
```

```
92 if( isempty(TS_theor_params120))
```

```
93 %% Define: Theoretical TS model parameters
```

```
94 TS_theor_params120.Ftminspec = 119.5; % kHz minimum✓  
frequency to model
```

```
95 TS_theor_params120.Ftmaxspec = 120.5; % kHz maximum✓  
frequency to model
```

```
96 TS_theor_params120.nF = 10; % number of✓  
frequencies to model
```

```
97 TS_theor_params120.sphereDiameter = sphereProp.sphereDiameter; %✓  
25.4; % sphere diameter (mm)
```

```
98 TS_theor_params120.sphereType = sphereProp.sphereType; %✓  
sphere type, '%WC' or 'CU'
```

```
99 TS_theor_params120.HZPASS = -0.1; % avoids TS samples✓  
affected by filter edges
```

```
100     TS_theor_params120.soundVel      = WaterProp.soundVel; %1500; %✓  
water sound speed  
101     TS_theor_params120.rhow          = WaterProp.density; % water✓  
density 1033.0  
102     TS_theor_params120.doPLOTTSBINNED = false;  
103 end  
104  
105  
106 %✓  
----- ✓  
-  
107 %% linear pulse-compressed amplitude and range, from raw IQ data  
108 %✓  
----- ✓  
-  
109 if(doPC)  
110     [pcLin, rangePC] = PulseCompression(Data, Config, WaterProp);  
111 else  
112     % [cwLin, r] = ProcRawCW(Data, Config);  
113     fprintf('Option to do pulse compression is false. Returning.✓  
\n');  
114     return;  
115 end  
116  
117 Nsamp = size(pcLin,1); % Number of samples == number of pings  
118  
119 pcLog = 10*log10(pcLin);  
120  
121 assignin('base', 'rangePC', rangePC);  
122  
123 % % % % Debugging, length of XMIT  
124 % % % Z0=Data.RawEcho1_90kHz_DataI+j*Data.RawEcho1_90kHz_DataQ;  
125 % % % Xmit0=Data.RawEcho1_90kHzTx_DataI+j*Data.✓  
RawEcho1_90kHzTx_DataQ;  
126 % % % N_XMIT = length(Xmit0);  
127  
128  
129  
130 %✓  
----- ✓  
-
```

```
131 %% Range, as recorded including Tx pulse
132 %✓
----- ✓
-
133 if Config.echo_frequency1 == 90 && Config.echo_pulseComp1 == 1
134     nEcho=1;
135 else
136     if isfield(Config,'echo_frequency2') && Config.echo_frequency2 ✓
== 90 && Config.echo_pulseComp2 == 1
137         nEcho=2;
138     else
139         if isfield(Config,'echo_frequency3') && Config. ✓
echo_frequency3 == 90 && Config.echo_pulseComp3 == 1
140             nEcho=3;
141         else
142             disp('No 90 kHz pulse compressed data');
143             return;
144         end
145     end
146 end
147 % eval(['sampleRate = Config.echo_rawSampleRate' num2str(nEcho) ✓
';'])
148 % eval(['nRawSamp    = Config.echo_nRawSamp'          num2str(nEcho) ✓
';'])
149 Range = ['Data.RawEcho' num2str(nEcho) '_90kHz_Range'];
150
151
152
153 %✓
----- ✓
-
154 %% Find range, index, and amplitude of sphere target
155 %✓
----- ✓
-
156 f_disp_dbstack_info()
157 fprintf('Find range, index, and amplitude of sphere target.\n');
158
159 % % approxDist = sphereProp.approxDist;
160 % % i=find((range>approxDist-0.5) & (range<approxDist+0.5));
161 % % spherePos.index = zeros(Nsamp,1);
```

```
162 % % spherePos.range = zeros(Nsamp,1);
163 % %
164 % % for k=1:Nsamp
165 % %     indc = i(find(pcLin(k,i) == max((pcLin(k,i)))));
166 % %     %spherePos(k) = r(j);
167 % %     spherePos.index(k,1) = indc;
168 % %     spherePos.range(k,1) = range(indc);
169 % %     spherePos.ampLin(k,1) = pcLin(k,indc);
170 % % end
171 % %
172 % % % spherpos.amp* are only used for plots
173 % % spherePos.ampLinmean = nanmean( spherePos.ampLin );
174 % % spherePos.ampLog      = 10*log10( spherePos.ampLin );
175 % % spherePos.ampLogmean = 10*log10( spherePos.ampLinmean );
176 % % spherePos.rangemean  = nanmean( spherePos.range );
177 % %
178 % % fprintf('nanmean(spherePos.index) = %.1f (sample)\n', nanmean(
(spherePos.index) );
179 % % fprintf('nanmean(spherePos.range) = %.2f (m)\n', nanmean(
(spherePos.range) );
180
181 % % Now, get sphere range and index by call to function✓
f_find_spherepos
182 rangebuffer = 0.5; % meters
183 %[spherePos90fm] = f_find_spherepos( pcLin, range, sphereProp.✓
approxDist, rangebuffer);
184 [spherePos90fm] = f_find_spherepos( Data, Config, pcLin, rangePC,✓
sphereProp.approxDist, rangebuffer);
185
186
187
188 if(doSHOWFIGS)
189     % Figures
190     rowoffset = 30;
191     rows2plot = floor(nanmean(spherePos90fm.index))-rowoffset :✓
floor(nanmean(spherePos90fm.index))+rowoffset ;
192
193     MSUB = 1;
194     NSUB = 4;
195     figure('Color',[1 1 1],'Name','(calibrateSig100)-Fig1. Sphere✓
location.');
```

```
196     subplot(MSUB,NSUB,1)
197     imagesc( pcLin' );
198     title('pcLin');
199     subplot(MSUB,NSUB,2)
200     imagesc( pcLog' );
201     title('pcLog');
202
203     subplot(MSUB,NSUB,3)
204     plot( max(pcLin,1), [1:size(pcLog,2)], '.-k'); %max(pcLin)
205     title('max(pcLin) by sample')
206     set (gca,'YDir','reverse')
207     ylim( [1, size(pcLog,2)] );
208
209     hold on;
210     plot( max(spherePos90fm.ampLin) , spherePos90fm.indexmean, ✓
'dg','MarkerSize',12,'LineWidth',2);
211
212     subplot(MSUB,NSUB,4)
213     plot( mode(pcLog,1), rangePC, '.-k');
214     title('mode(pcLog) by range')
215     set (gca,'YDir','reverse')
216     ylim( [0, max(rangePC)] );
217     % sphere expected location
218     hold on;
219     plot( 10*log10(spherePos90fm.ampLinmean) , spherePos90fm. ✓
rangemean, 'og','MarkerSize',12,'LineWidth',2);
220     text( 10*log10(spherePos90fm.ampLinmean+0.1), spherePos90fm. ✓
rangemean, 'target', 'color',[0, .8, 0]);
221
222     MSUB = 3;
223     NSUB = 1;
224     figure('Color',[1 1 1],'Name','(calibrateSig100)-Fig2. Sphere ✓
amp. ');
225     subplot(MSUB,NSUB,1)
226     plot( spherePos90fm.range , spherePos90fm.ampLin, '.-k');
227     subplot(MSUB,NSUB,2)
228     h1 = histogram(spherePos90fm.ampLin);
229     h1.Normalization = 'probability';
230     h1.NumBins = 15;
231     subplot(MSUB,NSUB,3)
232     h2 = histogram(spherePos90fm.ampLog);
```

```
233         h2.Normalization = 'probability';
234         h2.NumBins = 15;
235     end
236
237     pcLin_quantiles = quantile( reshape(pcLin,[],1), [0.01, 0.1, 0.5, ✓
0.9, 0.95, 0.99, 0.999]);
238     fprintf('pcLin quantiles\n')
239     fprintf('%.4f\t',pcLin_quantiles );
240     fprintf('\n');
241
242
243
244
245 %✓
----- ✓
-
246 %% Calculate theoretical TS of sphere for the broadband frequency ✓
bins
247 %✓
----- ✓
-
248 f_disp_dbstack_info()
249 fprintf('Calculate theoretical TS.\n');
250
251 % call modified 'binnedTargetStrength' code
252 % % TS_theor_paramsFM.Ftminspec      = 68.4;  % kHz minimum ✓
frequency to model
253 % % TS_theor_paramsFM.Ftmaxspec      = 113.4;  % kHz maximum ✓
frequency to model
254 % % TS_theor_paramsFM.nF             = 500;    % number of ✓
frequencies to model
255 % % TS_theor_paramsFM.sphereDiameter = 25.4;   % sphere diameter ✓
(mm)
256 % % TS_theor_paramsFM.sphereType     = 'WC';   % sphere type, %'WC' ✓
or 'CU'
257 % % TS_theor_paramsFM.HZPASS         = -0.1;   % avoids TS samples ✓
affected by filter edges
258 % % TS_theor_paramsFM.soundVel       = 1500;   % water sound speed
259 % % TS_theor_paramsFM.rhow           = 1033.0; % water density
260 % % doPLOTTSBINNED                   = false;
261 doFBINNED                           = true;
```

```
262
263 % [TS_theor_binned, TS_theor_params] = f_theorTargetStrengthBinned✓
(TS_theor_params, doPLOTTSBINNED);
264 [TS_theor_binned90fm] = f_theorTargetStrengthBinned✓
(TS_theor_paramsFM, TS_theor_paramsFM.doPLOTTSBINNED, doFBINNED);
265
266
267 %✓
-----✓
-
268 %% Calculate theoretical TS of sphere for the narrowband 70 kHz data
269 %✓
-----✓
-
270 %% Define: Theoretical TS model parameters
271 TS_theor_params70.Ftminspec      = 69.5; % kHz minimum frequency✓
to model
272 TS_theor_params70.Ftmaxspec      = 70.5; % kHz maximum frequency✓
to model
273 TS_theor_params70.nF             = 10;   % number of frequencies to✓
model
274 TS_theor_params70.sphereDiameter = sphereProp.sphereDiameter; %25.4;✓
% sphere diameter (mm)
275 TS_theor_params70.sphereType     = sphereProp.sphereType; % sphere✓
type, %'WC' or 'CU'
276 TS_theor_params70.HZPASS         = -0.1; % avoids TS samples✓
affected by filter edges
277 TS_theor_params70.soundVel       = WaterProp.soundVel; %1500; %✓
water sound speed
278 TS_theor_params70.rhow           = WaterProp.density; % water✓
density 1033.0
279
280 doFBINNED                        = false;
281
282 [TS_theor_binned70] = f_theorTargetStrengthBinned(TS_theor_params70, ✓
TS_theor_params70.doPLOTTSBINNED, doFBINNED);
283
284
285 %✓
-----✓
-
```

```
286 %% Calculate theoretical TS of sphere for the narrowband 120 kHz✓
data
287 %✓
-----✓
-
288 %% Define: Theoretical TS model parameters
289 TS_theor_params120.Ftminspec      = 119.5;  % kHz minimum frequency✓
to model
290 TS_theor_params120.Ftmaxspec      = 120.5;  % kHz maximum frequency✓
to model
291 TS_theor_params120.nF             = 10;     % number of frequencies✓
to model
292 TS_theor_params120.sphereDiameter = sphereProp.sphereDiameter; %✓
25.4; % sphere diameter (mm)
293 TS_theor_params120.sphereType     = sphereProp.sphereType; % sphere✓
type, %'WC' or 'CU'
294 TS_theor_params120.HZPASS         = -0.1;   % avoids TS samples✓
affected by filter edges
295 TS_theor_params120.soundVel       = WaterProp.soundVel; %1500; %✓
water sound speed
296 TS_theor_params120.rhow           = WaterProp.density; % water✓
density 1033.0
297
298 doFBINNED                         = false;
299
300 [TS_theor_binned120] = f_theorTargetStrengthBinned✓
(TS_theor_params120, TS_theor_params120.doPLOTTSBINNED, doFBINNED);
301
302
303 %% TS_theor_binned data to return
304 TS_theor_binned.TS_theor_binned90fm = TS_theor_binned90fm;
305 TS_theor_binned.TS_theor_binned70   = TS_theor_binned70;
306 TS_theor_binned.TS_theor_binned120  = TS_theor_binned120;
307
308
309
310
311 %✓
-----✓
-
312 %% Calculate Raw-Sample Broadband Echograms, from Raw IQ data
```

```
313 % Note - this can be done outside of this function and passed as✓
arg.
314 %✓
-----✓
-
315 if( doRAWSAMPLE90ECHOGRAM )
316     NBINF = [] % default nbinf (number of frequency bins) is 5,✓
defined in Config
317
318     % Struct Options should include values for:
319     pcegOptions.retRawData = 1 % 1 for testing
320     pcegOptions.NBINF = []; % empty value here means use default (5)✓
from Config
321
322     % Binned 90fm echogram
323     [echogramiq90fm,fEcho90fm,Range90fm,SampleBinned90fm] =✓
calcEchogram90kHzBinned(Config, Data, WaterProp, pcegOptions);
324     [echogramiq90fm,fEcho90fm,Range90fm,Sample90fm] =✓
calcEchogram90kHzBinned(Config, Data, WaterProp, pcegOptions);
325
326
327
328     %% DEBUGGING -
329     % % Compare
330     % [echogram90bin1,echogram90bin2,echogram90bin3,echogram90bin4,✓
echogram90bin5] = calcEchogram90kHzBinned_Orig(Config, Data);
331
332 end
333
334
335
336
337 %✓
-----✓
-
338 %% Calculate Range-Binned Broadband Echograms, from Raw IQ data
339 % Note - this can be done outside of this function and passed as✓
arg.
340 %✓
-----✓
-
```

```
341 NBINF = [] % default nbinf (number of frequency bins) is 5, defined✓
in Config
342
343 % Struct Options should include values for:
344 pcegOptions.retRawData = 0 % 1 for testing
345 pcegOptions.NBINF = []; % empty value here means use default (5)✓
from Config
346
347 % Binned 90fm echogram
348 %[echogramiq90fm,fEcho90fm,Range90fm,SampleBinned90fm] =✓
calcEchogram90kHzBinned(Config, Data, WaterProp, pcegOptions);
349 [echogramiq90fmBinned,fEcho90fm,Range90fmBinned,SampleBinned90fm] =✓
calcEchogram90kHzBinned(Config, Data, WaterProp, pcegOptions);
350
351
352 % % if( DEBUGEXPERIMENTAL )
353 % % % Call: x_debug_experimentalranges
354 % % end
355
356
357
358 %% FUTURE - create more frequency bins,
359 % % Note that this option is active and allows > 5 bins
360 % % by setting pcegOptions.NBINF = 9;
361 % % (empty pcegOptions.NBINF means use default (5) from Config)
362 % % and then calling calcEchogram90kHzBinned
363 % % [echogramiq90fm,fEcho,Range90fm] = calcEchogram90kHzBinned✓
(Config, Data, WaterProp, pcegOptions);
364 % % calcEchogram90kHzBinned then produces echogramiq90fm with more✓
bins,
365 % % but they still represent very wide bands.
366
367
368
369 %✓
-----✓
-
370 %% Calculate 70 kHz CW Echogram, from Raw IQ data
371 % and get stored 70 kHz amp echogram for comparison.
372 %✓
-----✓
```

```
-  
373 % [echogramiq70] = calcEchogram70kHz(Config, Data, WaterProp, []);  
374 [echogramiq70,fEcho70,Range70,pulse_length70] = calcEchogram70kHz✓  
(Config, Data, WaterProp, []);  
375  
376  
377 %✓  
----- ✓  
-  
378 %% Calculate 120 kHz CW Echogram, from Raw IQ data  
379 %   and get stored 120 kHz amp echogram for comparison.  
380 %✓  
----- ✓  
-  
381 [echogramiq120,fEcho120,Range120,pulse_length120] =✓  
calcEchogram120kHz(Config, Data, WaterProp, []);  
382  
383  
384  
385  
386 %✓  
----- ✓  
-  
387 %% Sphere position (spherePos) data for 90fmBinned, 70, & 120  
388 %   call function f_find_spherepos  
389 %✓  
----- ✓  
-  
390  
391  
392 %% TO DO - DEBUG THIS  
393 %% 90fm Binned  
394 rangebuffer = 1.25; % meters % def 0.4  
395  
396 myfields = fieldnames(echogramiq90fmBinned);  
397 myfield = myfields{3}; % TO DO - fix this lazy use of 3rd echogram  
398 echogram90x = echogramiq90fmBinned.(myfield);  
399 [spherePos90fmBinned] = f_find_spherepos( Data, Config, echogram90x,✓  
Range90fmBinned, sphereProp.approxDist, rangebuffer);  
400  
401 % % %% DEBUGGING
```

```
402 % % if(DEBUGEXPERIMENTAL)
403 % %     util_debugsphereposfor90fmBinned
404 % % end
405
406
407
408 %% 70cwBinned
409 rangebuffer = 2.0; % meters
410
411 %[spherePos70] = f_find_spherepos( echogramiq70, Range70, ✓
sphereProp.approxDist, rangebuffer);
412 [spherePos70] = f_find_spherepos( Data, Config, echogramiq70, ✓
Range70, sphereProp.approxDist, rangebuffer);
413
414
415 %% 120cwBinned
416 % % OFFSETDIST120 = -2.4;
417 % % OFFSETDIST120 = 0;
418 approxDist120 = sphereProp.approxDist + OFFSETDIST120;
419 fprintf('\n[!] Warning: approxDist120=sphereProp.approxDist + %.2f ✓
m. [!]\n',OFFSETDIST120)
420 %[spherePos120] = f_find_spherepos( echogramiq120, Range120, ✓
approxDist120 , rangebuffer);
421 [spherePos120] = f_find_spherepos( Data, Config, echogramiq120, ✓
Range120, approxDist120 , rangebuffer);
422
423 % sphere position figures for 70 & 120
424 if(DEBUGGING)
425     f_plot_echograms( echogramiq70, echogramiq70 , Range70 , ✓
{'70','70'} );
426     hold on;
427     plot( spherePos70.ampLinmean, spherePos70.rangemean, ✓
'og','MarkerSize',12,'LineWidth',2)
428     newfigname=[get(gcf,'Name') ' spherepos 70kHz'];
429     set(gcf,'Name',newfigname);
430
431     f_plot_echograms( echogramiq120, echogramiq120 , Range120 , ✓
{'120','120'} );
432     hold on;
433     plot( spherePos120.ampLinmean, spherePos120.rangemean, ✓
'og','MarkerSize',12,'LineWidth',2)
```

```
434     newfigname=[get(gcf,'Name') ' spherepos 120kHz'];
435     set(gcf,'Name',newfigname);
436
437     f_plot_echograms( echogram90x, echogram90x ,   Range90fmBinned , ✓
{'90fmbinned','90fmbinned'} );
438     hold on;
439     plot( spherePos90fmBinned.ampLinmean, spherePos90fmBinned. ✓
rangemean, 'og','MarkerSize',12,'LineWidth',2)
440     newfigname=[get(gcf,'Name') ' spherepos 120kHz'];
441     set(gcf,'Name',newfigname);
442 end
443
444
445 % % ✓
----- ✓
-
446 % %% DEBUGGING - assignin all vars
447 % %   call function
448 % % ✓
----- ✓
-
449 % if( DEBUGGING )
450 %     f_assignin_allvars;
451 %
452 %     save('calibrateSig100wkspc.mat')
453 %     disp('L354. Saved workspace. Returning for debugging.');
```

```
454 %
455 %     return;
456 % end
457
458
459
460
461 % ✓
----- ✓
-
462 %% Calculate Absorption, by frequency
463 %   use WaterProp
464 % ✓
----- ✓
-
```

```
465 ABSORPTION90FM = [];
466 ABSORPTION70CW = [];
467 ABSORPTION120CW = [];
468
469 % freq = frequency in Hz
470 % Z = depth in km (yes, kilometers)
471 % T = temperature in C
472 % S = salinity in ppt
473 % pH = pH
474 absorp.Z = 0.001; % (km)
475 absorp.T = WaterProp.temp; % default 13.0; % (deg C)
476 absorp.S = WaterProp.sal; % default 33.0; (psu)
477 absorp.pH = 8.0; % Approx.
478
479 %% Absorption 90fm
480 fprintf('freq (kHz)\taborption (dB/m)\n');
481 for( indx = 1 : length(fEcho90fm))
482     f_Hz = fEcho90fm(indx); % frequency (Hz) e.g. 70000
483     absorp.freq = f_Hz;
484     [alpha] = absorption(absorp.freq, absorp.Z, absorp.T, absorp.S, ↵
absorp.pH);
485     ABSORPTION90FM(indx) = alpha * 1e-3;
486     fprintf('%.1f\t%.4f\n', f_Hz*1e-3, ABSORPTION90FM(indx) );
487 end
488
489 fprintf('freq (kHz)\taborption (dB/m)\n');
490 for( indx = 1 : length(fEcho70))
491     f_Hz = fEcho70(indx); % frequency (Hz) e.g. 70000
492     absorp.freq = f_Hz;
493     [alpha] = absorption(absorp.freq, absorp.Z, absorp.T, absorp.S, ↵
absorp.pH);
494     ABSORPTION70CW(indx) = alpha * 1e-3;
495     fprintf('%.1f\t%.4f\n', f_Hz*1e-3, ABSORPTION70CW(indx) );
496 end
497
498 fprintf('freq (kHz)\taborption (dB/m)\n');
499 for( indx = 1 : length(fEcho120))
500     f_Hz = fEcho120(indx); % frequency (Hz) e.g. 70000
501     absorp.freq = f_Hz;
502     [alpha] = absorption(absorp.freq, absorp.Z, absorp.T, absorp.S, ↵
absorp.pH);
```

```
503     ABSORPTION120CW(indx) = alpha * 1e-3;
504     fprintf('%0.1f\t%0.4f\n', f_Hz*1e-3, ABSORPTION120CW(indx) );
505 end
506
507
508
509
510
511 %✓
----- ✓
-
512 %% Convert echogram amplitude to target strength, TS_meas
513 %✓
----- ✓
-
514 CALGAIN90fm = [];
515 CALGAIN70    = [];
516 CALGAIN120   = [];
517
518 %% TS90fmbinned (TS measured)
519 echogramraw  = echogramiq90fmBinned;
520 fEcho        = fEcho90fm;
521
522 % %Range      = Range90fm;
523 Range        = Range90fmBinned;
524
525 soundVel     = WaterProp.soundVel;
526 % pulse_length = 5e-3;
527 pulse_length = [];
528
529 % [TS, Sv, TStype] = f_TS_from_Sig100_amp(Config, Data, echogram, ✓
fEcho, Range, CALGAIN)
530 [TS90fm, Sv90fm, TStype90fm, RangeTS90fm] = f_TS_from_Sig100_amp ✓
(Config, Data, ...
531     pulse_length, echogramraw, fEcho, ABSORPTION90FM, Range, ✓
soundVel, CALGAIN90fm);
532
533     % rename figure
534     set(gcf, 'Name', '(f_TS_from_Sig100_amp)-TS90fm');
535
536 clear echogramraw fEcho Range pulse_length;
```

```
537
538 %XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
539 % % %DEBUGGING call: x_for_comparison_TS90fm_from_recorded.m
540 %XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
541
542
543
544 %% TS70 (TS measured)
545 echogramraw = echogramiq70;
546 fEcho       = fEcho70;
547 Range       = Range70; % Range from calcEchogram70kHz
548 % pulse_length = 1e-3;
549 pulse_length = pulse_length70;
550
551 [TS70, Sv70, TStype70, RangeTS70] = f_TS_from_Sig100_amp(Config, ✓
Data,...
552     pulse_length, echogramraw, fEcho, ABSORPTION90FM, Range, ✓
soundVel, CALGAIN70);
553
554 % rename figure
555 set(gcf, 'Name', '(f_TS_from_Sig100_amp)-TS70');
556
557 clear echogramraw fEcho Range pulse_length;
558
559
560
561 %% TS120 (TS measured)
562 echogramraw = echogramiq120;
563 fEcho       = fEcho120;
564 Range       = Range120; % Range from calcEchogram120kHz
565 % pulse_length = 1e-3;
566 pulse_length = pulse_length120;
567
568 [TS120, Sv120, TStype120, RangeTS120] = f_TS_from_Sig100_amp(Config, ✓
Data,...
569     pulse_length, echogramraw, fEcho, ABSORPTION90FM, Range, ✓
soundVel, CALGAIN120);
570
571 % rename figure
572 set(gcf, 'Name', '(f_TS_from_Sig100_amp)-TS120');
573
```

```
574 clear echogramraw fEcho Range pulse_length;
575
576
577
578
579
580
581 %✓
----- ✓
-
582 %% Get TS_meas at samples of sphere position
583 %   TS_meas_binned from echogramiq90fm, echogramiq70, &✓
echogramiq120
584 %   spherePos.index
585 %✓
----- ✓
-
586 %% TS_meas_targ 90fm
587 % doTARGOFFSET = true;
588 % TARGSAMPOFFSET = 43;
589 NCELLBUFF = 1;
590
591 % % %% TS_meas 90fm NOT BINNED
592 % % [TS_meas_target90fm, TS_meas_target_mean90fm] =✓
f_get_TS_meas_of_target( fEcho90fm, TS90fm, Range90fm, spherePos90fm, ✓
doTARGOFFSET, TARGSAMPOFFSET, NCELLBUFF);
593 % % newfigname=[get(gcf,'Name') ' 90fm (NOT BINNED)'];
594 % % set(gcf,'Name',newfigname);
595
596
597 %% TS_meas_targ 90fm Binned
598 TARGSAMPOFFSETBINNED = 0;
599 [TS_meas_target90fm, TS_meas_target_mean90fm] =✓
f_get_TS_meas_of_target(...
600 fEcho90fm, TS90fm, Range90fmBinned, spherePos90fmBinned, ✓
doTARGOFFSET, TARGSAMPOFFSETBINNED, NCELLBUFF);
601 %
602 newfigname=[get(gcf,'Name') ' 90fm'];
603 set(gcf,'Name',newfigname);
604
605
```

```

606
607 %% TS_meas_targ 70
608 doTARGOFFSET = true;
609 TARGSAMPOFFSET = 0;
610 NCELLBUFF = 1;
611 [TS_meas_target70, TS_meas_target_mean70] = f_get_TS_meas_of_target✓
(...
612     fEcho70, TS70, Range70, spherePos70, doTARGOFFSET, ✓
TARGSAMPOFFSET, NCELLBUFF);
613 %
614 newfigname=[get(gcf, 'Name') ' 70 kHz'];
615 set(gcf, 'Name', newfigname);
616
617 %% TS_meas_targ 120
618 doTARGOFFSET = true;
619 TARGSAMPOFFSET = 0;
620 NCELLBUFF = 1;
621 [TS_meas_target120, TS_meas_target_mean120] = ✓
f_get_TS_meas_of_target(...
622     fEcho120, TS120, Range120, spherePos120, doTARGOFFSET, ✓
TARGSAMPOFFSET, NCELLBUFF);
623 % hold on;
624 % plot( spherePos120.ampLinmean, spherePos120.rangemean, ✓
'og', 'MarkerSize', 12, 'LineWidth', 2)
625 newfigname=[get(gcf, 'Name') ' 120 kHz'];
626 set(gcf, 'Name', newfigname);
627
628 clear doTARGOFFSET TARGSAMPOFFSET;
629
630
631
632
633
634 % ✓
----- ✓
-
635 %% CALIBRATION
636 % Calc TS measured for each freq band, and get d(TS_meas - ✓
TS_theor).
637 % TS meas == Echogram data from calcEchogram#.
638 % Store in struct calibration to return.

```

```

639 %✓
----- ✓
-
640
641 %✓
----- ✓
-
642 % % Print filename prefix, from Config, and firmware version
643 %✓
----- ✓
-
644 fprintf✓
('\n\n-----\n');
645 fprintf('CALIBRATION RESULTS\n');
646 fprintf✓
('-----\n');
647 fprintf('File: %s\n',Config.File_file_prefix);
648 fprintf('Firmware: %d\n',Config.fwVersionDoppler);
649
650 if(doCALIBRATE90FM)
651     fprintf('90 kHz FM\n');
652     % TS_theoretical
653     for( indx = 1 : length(fEcho90fm))
654         fprintf('TS_theo(f=%.2f) = %.2f\n',...
655             fEcho90fm(indx),...
656             TS_theor_binned.TS_theor_binned90fm.TS_theor_mean_bin✓
(indx));
657     end
658     fprintf('\n');
659
660     % CALGAIN = TS_meas - TS_theor;
661     for( indx = 1 : length(fEcho90fm))
662
663         TS_theor = TS_theor_binned.TS_theor_binned90fm.✓
TS_theor_mean_bin(indx);
664
665         TS_meas = TS_meas_target_mean90fm(indx);
666
667         CALGAIN90fm(indx) = TS_theor - TS_meas;
668
669         fprintf('CALGAIN(f=%.2f) = %.2f\n', fEcho90fm(indx),✓

```

```
CALGAIN90fm(indx));
670     clear TS_theor TS_meas;
671 end
672 end
673
674 if(doCALIBRATE70)
675     fprintf('70 kHz\n');
676     % TS_theoretical
677     for( indx = 1 : length(fEcho70))
678         fprintf('TS_theo(f=%.2f) = %.2f\n',...
679             fEcho70(indx),...
680             TS_theor_binned.TS_theor_binned70.TS_theor_mean_bin✓
681         (indx));
682     end
683     fprintf('\n');
684     % CALGAIN = TS_meas - TS_theor;
685     for( indx = 1 : length(fEcho70))
686
687         TS_theor = TS_theor_binned.TS_theor_binned70.✓
688         TS_theor_mean_bin(indx);
689
690         TS_meas = TS_meas_target_mean70(indx);
691
692         CALGAIN70(indx) = TS_theor - TS_meas;
693
694         clear TS_theor TS_meas;
695         fprintf('CALGAIN(f=%.2f) = %.2f\n', fEcho70(indx), CALGAIN70✓
696     (indx));
697 end
698 end
699 %% Cal 120
700 if(doCALIBRATE120)
701     fprintf('120 kHz\n');
702     % TS_theoretical
703     for( indx = 1 : length(fEcho120))
704         fprintf('TS_theo(f=%.2f) = %.2f\n',...
705             fEcho120(indx),...
706             TS_theor_binned.TS_theor_binned120.TS_theor_mean_bin✓
707     (indx));
```

```

706     end
707     fprintf('\n');
708
709     % CALGAIN = TS_meas - TS_theor;
710     for( indx = 1 : length(fEcho120))
711
712         TS_theor = TS_theor_binned.TS_theor_binned120.✓
TS_theor_mean_bin(indx);
713
714         TS_meas = TS_meas_target_mean120(indx);
715
716         CALGAIN120(indx) = TS_theor - TS_meas;
717         clear TS_theor TS_meas;
718         fprintf('CALGAIN(f=%.2f) = %.2f\n', fEcho120(indx),✓
CALGAIN120(indx));
719     end
720 end
721
722
723
724 fprintf✓
('-----\n');
725
726
727 % % Recall: TS_theor_binned is a struct with fields for each mode
728 % % % TS_theor_binned.TS_theor_binned90fm = TS_theor_binned90fm;
729 % % % TS_theor_binned.TS_theor_binned70 = TS_theor_binned70;
730 % % % TS_theor_binned.TS_theor_binned120 = TS_theor_binned120;
731
732
733 %% DATA TO RETURN FROM FUNCTION
734 calibration.fm90.CALGAIN = CALGAIN90fm;
735 calibration.fm90.TS_theor_binned = TS_theor_binned.✓
TS_theor_binned90fm;
736 calibration.fm90.fEcho = fEcho90fm;
737 calibration.fm90.TS_meas_target = TS_meas_target90fm;
738 calibration.fm90.TS_meas_target_mean = TS_meas_target_mean90fm;
739
740 calibration.nb70.CALGAIN = CALGAIN70;
741 calibration.nb70.TS_theor_binned = TS_theor_binned.✓
TS_theor_binned70;

```

```

742 calibration.nb70.fEcho = fEcho70;
743 calibration.nb70.TS_meas_target = TS_meas_target70;
744 calibration.nb70.TS_meas_target_mean = TS_meas_target_mean70;
745
746 calibration.nb120.CALGAIN = CALGAIN120;
747 calibration.nb120.TS_theor_binned = TS_theor_binned.✓
TS_theor_binned120;
748 calibration.nb120.fEcho = fEcho120;
749 calibration.nb120.TS_meas_target = TS_meas_target120;
750 calibration.nb120.TS_meas_target_mean = TS_meas_target_mean120;
751
752
753 %✓
-----✓
-
754 %% Return: spherePos,TS_theor_binned,calibration
755 %✓
-----✓
-
756
757 fprintf('\n\n*****\n');
758 fprintf('Function calibrateSig100 completed.\n');
759 fprintf('*****\n\n');
760
761 % end % %// end function calibrateSig100
762
763
764
765 %% FUNCTIONS
766 % % % function f_find_spherepos
767 % % function [spherePos] = f_find_spherepos( Data, Config, echogram,✓
range, approxDist, rangebuffer)
768 % % %function [spherePos] = f_find_spherepos( echogram, range,✓
approxDist, rangebuffer)
769 % %     range = range(1,:);
770 % %     Nsamp = size(echogram,1); % Number of samples == number of✓
pings
771 % %
772 % %     if isempty(rangebuffer)
773 % %         rangebuffer = 0.5; % m
774 % %     end

```

```

775 % %
776 % %     spherePos.index = zeros(Nsamp,1);
777 % %     spherePos.range = zeros(Nsamp,1);
778 % %
779 % %     %✓
-----✓
-
780 % %     %% Find range, index, and amplitude of sphere target
781 % %     %✓
-----✓
-
782 % %     f_disp_dbstack_info()
783 % %     fprintf('Find range, index, and amplitude of sphere target.✓
\n');
784 % %
785 % %     approxDist = approxDist;
786 % %     i=find((range>approxDist-rangebuffer) &✓
(range<approxDist+rangebuffer));
787 % %     spherePos.index = zeros(Nsamp,1);
788 % %     spherePos.range = zeros(Nsamp,1);
789 % %
790 % %     for k=1:Nsamp
791 % %         indc = i(find(echogram(k,i) == max((echogram(k,i)))));
792 % %         %spherePos(k) = r(j);
793 % %         spherePos.index(k,1) = indc;
794 % %         spherePos.range(k,1) = range(indc);
795 % %         spherePos.ampLin(k,1) = echogram(k,indc);
796 % %     end
797 % %
798 % %     spherePos.indexmean = nanmean( spherePos.index );
799 % %     spherePos.ampLinmean = nanmean( spherePos.ampLin );
800 % %     spherePos.rangemean = nanmean( spherePos.range );
801 % %     spherePos.ampf_find_sphereposLog = 10*log10( spherePos.✓
ampLin );
802 % %     spherePos.ampLogmean = 10*log10( spherePos.ampLinmean );
803 % %
804 % %     fprintf('nanmean(spherePos.index) = %.1f (sample)\n',✓
nanmean(spherePos.index) );
805 % %     fprintf('nanmean(spherePos.range) = %.2f (m)\n', nanmean✓
(spherePos.range) );
806 % % end

```

```

807
808
809 % % %% function f_get_TS_meas_of_target
810 % % function [TS_meas_target,TS_meas_target_mean] =✓
f_get_TS_meas_of_target( fEcho_binned, TS_binned, Range, spherePos,✓
doTARGOFFSET, TARGSAMPOFFSET, NCELLBUFF)
811 % %         if(isempty(doTARGOFFSET))
812 % %             doTARGOFFSET=false;
813 % %         end
814 % %         if(isempty(NCELLBUFF))
815 % %             NCELLBUFF = 1;
816 % %         end
817 % %
818 % %         fprintf('\n');
819 % %         % loop through binned frequencies
820 % %         TS_meas_target_mean = zeros(size(fEcho_binned,1));
821 % %         % % %         for( indx = 1 : length(fEcho_binned))
822 % %         % % %
823 % %         % % %             if(isstruct(TS_binned))
824 % %         % % %                 sfields = fieldnames(TS_binned);           %✓
get fields of TS struct
825 % %         % % %                 myfield = sfields{indx};           %✓
current field
826 % %         % % %                 TS_meas_band = TS_binned.(myfield); %✓
TS_meas samples for this f-band
827 % %         % % %                 else
828 % %         % % %                 TS_meas_band = TS_binned;
829 % %         % % %                 end
830 % %         % % %                 if(doTARGOFFSET)                         % adjust✓
for sphere pos from 90fm pulse compressed range
831 % %         % % %                 targindx = round(spherePos.index) +✓
TARGSAMPOFFSET;
832 % %         % % %                 TS_meas_band_itarget = TS_meas_band( : ,✓
targindx-NCELLBUFF : targindx+NCELLBUFF );
833 % %         % % %                 else
834 % %         % % %                 targindx = round(spherePos.index);
835 % %         % % %                 TS_meas_band_itarget = TS_meas_band( : ,✓
targindx-NCELLBUFF : targindx+NCELLBUFF );
836 % %         % % %                 end
837 % %         % % %                 if(isstruct(TS_binned))
838 % %         % % %                 TS_meas_target.(myfield) =✓

```

```

TS_meas_band_itarget;
839 % % % % else
840 % % % % TS_meas_target = TS_meas_band_itarget;
841 % % % % end
842 % % % %
843 % % % % if(isstruct(TS_binned))
844 % % % % TS_meas_target_q = quantile(✓
TS_meas_target.(myfield), [0.95, 0.99, 0.999]);
845 % % % % else
846 % % % % TS_meas_target_q = quantile(✓
TS_meas_target, [0.95, 0.99, 0.999]);
847 % % % % end
848 % % % % TS_meas_target_mean(indx) = 10*log10( mean(10.✓
^(reshape(TS_meas_target_q,[],1)/10)));
849 % % % % fprintf('(findex %d) TS_meas_target_mean = %.✓
2f.\n', indx, TS_meas_target_mean(indx));
850 % % % %
851 % % % % %% Figure✓
=====
852 % % % % matnames = {'TS_meas_band'} ;
853 % % % % f_plot_echogram_maxamp( TS_meas_band , Range , ✓
matnames);
854 % % % % hold on;
855 % % % % plot( mean(TS_meas_band(:,mean(spherePos.✓
index))), mean(Range(mode(spherePos.index))),...
856 % % % % 'dm','MarkerSize',10,'LineWidth',1.1);
857 % % % % hold on;
858 % % % % % plot with NO OFFSET
859 % % % % hold on;
860 % % % % plot( TS_meas_target_mean(indx) , mean(Range✓
(mode(spherePos.index)+0)),...
861 % % % % 'xr','MarkerSize',10,'LineWidth',1.1);
862 % % % % % plot with TARGETOFFSET
863 % % % % hold on;
864 % % % % plot( TS_meas_target_mean(indx) , mean(Range✓
(mode(spherePos.index)+TARGSAMPOFFSET)),...
865 % % % % 'og','MarkerSize',10,'LineWidth',1.5);
866 % % % %
867 % % % % newfigname=[get(gcf,'Name') '✓
(f_get_TS_meas)'];
868 % % % % set(gcf,'Name',newfigname);

```

```

869 % %           % % %           %✓
=====
870 % %           % % %           end
871 % %           for( indf = 1 : length(fEcho_binned))
872 % %
873 % %           if(isstruct(TS_binned))
874 % %           sfields = fieldnames(TS_binned);           % get fields✓
of TS struct
875 % %           myfield = sfields{indf};           % current field
876 % %           TS_meas_band = TS_binned.(myfield); % TS_meas✓
samples for this f-band
877 % %           else
878 % %           TS_meas_band = TS_binned;
879 % %           end
880 % %           if(doTARGOFFSET)           % adjust for sphere✓
pos from 90fm pulse compressed range
881 % %           targindx = round(spherePos.index) + TARGSAMPOFFSET;
882 % %           TS_meas_band_itarget = TS_meas_band( : , targindx-✓
NCELLBUFF : targindx+NCELLBUFF );
883 % %           else
884 % %           targindx = round(spherePos.index);
885 % %           TS_meas_band_itarget = TS_meas_band( : , targindx-✓
NCELLBUFF : targindx+NCELLBUFF );
886 % %           end
887 % %           if(isstruct(TS_binned))
888 % %           TS_meas_target.(myfield) = TS_meas_band_itarget;
889 % %           else
890 % %           TS_meas_target = TS_meas_band_itarget;
891 % %           end
892 % %
893 % %           if(isstruct(TS_binned))
894 % %           TS_meas_target_q = quantile( TS_meas_target.✓
(myfield), [0.95, 0.99, 0.999]);
895 % %           else
896 % %           TS_meas_target_q = quantile( TS_meas_target, [0.95, ✓
0.99, 0.999]);
897 % %           end
898 % %           TS_meas_target_mean(indf) = 10*log10( mean(10.^(reshape ✓
(TS_meas_target_q, [], 1)/10)));
899 % %           fprintf('(findex %d) TS_meas_target_mean = %.2f.\n', ✓
indf, TS_meas_target_mean(indf));

```

```
900 % %
901 % %           %% Figure =====
902 % %           matnames = {'TS_meas_band'} ;
903 % %           f_plot_echogram_maxamp( TS_meas_band , Range , ✓
matnames);
904 % %           %
905 % %           % plot with NO OFFSET
906 % %           hold on;
907 % %           plot( mean(TS_meas_band(:,mode(spherePos.index))), mean ✓
(Range(mode(spherePos.index))),...
908 % %           'om','MarkerSize',10,'LineWidth',0.8);
909 % %           plot( mean(TS_meas_band(:,mode(spherePos.index))), mean ✓
(Range(mode(spherePos.index))),...
910 % %           'xm','MarkerSize',10,'LineWidth',0.8);
911 % %           hold on;
912 % % %           plot( TS_meas_target_mean(indf) , mean(Range(mode ✓
(spherePos.index)+0)),...
913 % % %           'xm','MarkerSize',10,'LineWidth',0.8);
914 % %           %
915 % %           % plot with TARGETOFFSET
916 % %           hold on;
917 % %           plot( TS_meas_target_mean(indf) , mean(Range(mode ✓
(spherePos.index)+TARGSAMPOFFSET)),...
918 % %           'og','MarkerSize',10,'LineWidth',1.2);
919 % %
920 % %           newfigname=[get(gcf,'Name') ' (f_get_TS_meas)'];
921 % %           set(gcf,'Name',newfigname);
922 % %           %=====
923 % %           end
924 % %           fprintf('\n');
925 % % end
926
```